Chapter 5
Selection Statements

CSE 4107 : Structured Programming I
REYNEP

5.0. Statements

So far, we've used
- *expression statements*
- *return statements*

Other C statements:
- **Compound** *statements*
- *null statements*

Most of C's remaining statements fall into three categories:
- **Selection** *statements:* `if` `switch`
- **Iteration** *statements:* `while` `do` `for`
- **Jump** *statements:* `break` `continue` `goto` `return`

5.1.1. boolean Expressions

```c
int main(void) {
    int i = 0, j = 20;

    if (i < j) {
        printf(" true: i is less than j");
    }
    else {
        printf("false: i is not less than j");
    }
}
```

- *A* **true** *value would indicate that i is less than j*

`if`
- *tests the value of an expression to see if it's* **"true"**
  - *being* **true** $\implies$ *processor gets inside the* `if` *-block*
  - *being* **false** $\implies$ *processor gets inside the* `else` *-block*

```c
if (true) {
    printf("We are inside the true block");
}
```

- *in C, the words* `true` `false` *are inside a file called* `stdbool.h`

```c
#include <stdio.h>


int main(void)
{
    if (1) {
        printf(" true: ");
    }
    else {
        printf("false: ");
    }
}
```

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{                   /* stdbool.h --> words: 'true', 'false' */
    if (true) {
        printf(" true: this will be printed");
    }
    else {
        printf("false: this won't be printed");
    }
}
```

- *in C*
  - `1` $=$ `true`
  - `0` $=$ `false`
  - `if (i < j)` $\implies$ `if (1)` *for our example case above*
    - *bcz,* `i=0` , `j=20`

- *in fact, inside* `stdbool.h` *, you can find*
  - `#define true 1`
  - `#define false 0`

-
  - *it's kinda like*
    - *the C compiler's dictionary has the words say* `int` `float` `double` `char` `void` , `return` , `if` `else`
    - *the* `stdio.h` *file has the words/definitions of say* `printf()` , `scanf()`
    - *the C compiler's dictionary doesn't have the word* `true` `false`
    - *the* `stdbool.h` *file adds these words*

Let me firmly, introduce another **new data-type** $\rightarrow$ `bool`

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    bool result = isPrime(50);

    if (result == true) {
        printf("50 is prime");
    }
    else {
        printf("50 is not prime");
    }
}
```

- *Other programming languages* `python` `java` `C++`
  - *has* `bool` *as* **built-in data type**
- *But in C*
  - `stdbool.h` **defines** *it*
    - *However, the story behind "How it's def." is kinda long*
      - ***https://stackoverflow.com/a/47822105***
- *in those other programming languages,*
  - *an expression such as* `i<j` *would have a* `bool` */logical data-type*
- *But in* `C`
  - *such comparisons yield an integer i.e.* `0` *or* `1`

5.1.2. Operators

Binary (two operands) Arithmetic Operators:

```
addition                int x = 10 + 5;
subtraction             int x = 10 - 5;
multiplication          int x = 10 * 5;
division                int x = 10 / 5;
remainder = modulus     int x = 10 % 5;
```

```
float a =    9 + 2.5f;      // a = 11.5
float b = 6.7f / 2;         // b = 3.35
  int c =   10 % 3;         // c = 1
  int d =   12 % 4;         // d = 0
```

Unary (one operands) Arithmetic Operators:

```
                        int y =  7;
unary plus              int x = +y;     int m = +1;
unary minus             int x = -y;     int n = -2;
```

Simple Assignment Operator

```
i = 5;
j = i;
k = 10*i + j;
```

```
int i, j, k;
i = j = k = 0;
```

[in/de]crement operators

```
i++;
i--;
```

- **post-[in/de]crement**

```
++i;
--i;
```

- **pre-[in/de]crement**

Compound Assignment Operator

```
i += 6      //      i = i + 2;
i -= 5      //      i = i - 2;
i *= 4      //      i = i * 2;
i /= 3      //      i = i / 2;
i %= 2      //      i = i % 2;
```

Relational Operators

```
5 < 10          // true
5 > 10          // false
101 <= 102      // true
101 >= 102      // false
```

Equality Operators

```
101 == 101      // true
101 != 101      // false
```

ivan's @ page 04

- *These operators produce* `0` *(false) or* `1` *(true) when used in expressions, say inside* `if ()`
- *These operators can be used to compare*
  - *integers*
  - *floating-point numbers,*
  - *operands of mixed types*
- *The* **precedence** *of the* **relational** *operators is lower than that of the* **arithmetic** *operators*
  - *e.g.* `i + j < k - 1` $\implies$ `(i + j) < (k - 1)`

ivan's @ page 05

- *The* **relational** *operators are left associative*

```
i < j < k
```

  - *is legal*
  - *but, does not test, whether* `j` **lies between** `i` **and** `k`

```
(i < j) < k
```

  - *Since the* `<` *operator is left associative, it ends up like this, above.*
    - *The 1 or 0 produced by* `i < j` *is then compared to* `k`

- `j` **lies between** `i` **and** `k`
  - *The correct expression is* `i<j && j<k`
- *The* **Equality** *Operators*
  - *are also left associative*
  - *also produce either 0 (false) or 1 (true) as their result*
  - *have* **lower precedence** *than the* **relational** *operators*
    - *So, The expression*
      - `i < j == j < k` *is equivalent to,* `(i < j) == (j < k)`

Logical Operators

```
bool a = true;
bool b = !a;        // false;
bool c = a && b;    // false;
bool d = a || b;    // true;
```

ivan's @ page 08

- *The* `!` *operator is* **unary**
- `&&` *and* `||` *operators are* **binary**.
- *The* **logical** *operators*
  - *produce* `0` *or* `1` *as their result*
  - *treat any non-zero operand as a* `true` *value and any zero operand as a* `false` *value*

ivan's @ page 09

- *Behavior of the logical operators:*
  - `!expr` *has the value* `1` *if* `expr` *has the value* `0`
  - `expr1 && expr2` *has the value* `1` *if the values of* `expr1` *and* `expr2` *are both nonzero*
  - `expr1 || expr2` *has the value* `1` *if either* `expr1` *or* `expr2` *(or both) has a nonzero*
  - *In all other cases, these operators produce the value 0*

"short-circuit" evaluation

- `&&` `||`
  - *first evaluate the* **left** *operand,*
  - *then the* **right** *one*

```
if ((i != 0) && (j/i > 0))    { printf("we do something in this block"); }
```

- *If the value of the expression can be deduced from the left operand alone, the right operand isn't evaluated*
  - `(i != 0)` *is evaluated first.*
    - **If,** *this part is* **true,** *only then the right part is checked. (notice we used* `&&` *)*
    - **If,** *this part is* **false,** *the entire expression must be false, so there's no need to evaluate* `(j/i > 0)`
      - *Without short-circuit evaluation, division by zero would have occurred, in this case*

"short-circuit" evaluation (continued)

```
if (i > 0 && ++j > 0)
{

}
```

- *If* `i > 0` *is* `false`*, then* `++j > 0` *is not evaluated, so* `j` *isn't incremented*
- *Side effects in logical expressions may not always occur because of the short-circuit nature*
- *The problem can be fixed by changing the condition to*

```
if (++j > 0 && i > 0) {}
```

- *or, even better, by incrementing j separately*

ivan's @ page12 : Precedence

- *The* `!` *operator*
  - *has the same precedence as the* **unary** *plus and minus operators*
  - *is right associative*
- *The* `&&` *and* `||` *operators*
  - `&&` *and* `||` *are left associative*
  - *The precedence of* `&&` *and* `||` *is lower than that of the relational and equality operators*

```
i < j && k == m
```

$\Longrightarrow$

```
(i < j) && (k == m)
```

5.1.3. Operators : all at once

Assignment Operators

Simple

```
i = 5;
j = i;
k = 10*i + j;
```

```
int i, j, k;
i = j = k = 0;
```

Compound

```
i += 6      //      i = i + 2;
i -= 5      //      i = i - 2;
i *= 4      //      i = i * 2;
i /= 3      //      i = i / 2;
i %= 2      //      i = i % 2;
```

```
i++;
i--;
```

- **post**

```
++i;
--i;
```

- **pre**

Boolean Operators

Relational Operators

```
5 < 10        // true
5 > 10        // false
101 <= 102    // true
101 >= 102    // false
```

Logical Operators

```
bool a = true;
bool b = !a;        // false;
bool c = a && b;    // false;
bool d = a || b;    // true;
```

Equality Operators

```
101 == 101    // true
101 != 101    // false
```

5.2.1. The `if` Statement

In its simplest form, the if statement has the form:

```
//
if (expression) {
    statements;
}
```

```
bool b = isPrime(50);
if (b == false) {
    printf("50 is prime");
}
```

- *When an* `if` *statement is executed,*
  - **expression** *is evaluated;*
    - `If` *that's value is nonzero,*
      - *then* **statements** *inside the curly barces are executed*

Possible Confusion

```
int  x = 50;            // 50;
bool b = (x == 50);     // true
int  b = (x == 50);     // 1
```

- *Notice that,* `==` *and* `=` *are different*
- *Confusing* `==` *(equality) with* `=` *(assignment) is perhaps the most common* C *prog. error*

```
if(i == 0) {}
```

- *tests whether* `i` $=$ `0`

```
if(i = 0) {}
```

- *assigns* `0` *to* `i`*, then tests whether the result is nonzero*

ivan's @ page15 : Multiple Statements

```
if (line_num == MAX_LINE_NUM)
{
    page_num++;
    line_num = 0;
}
```

- **ivan** *calls this* **Compound** *Statements*
- *A* **compound** *statement has the form* `{ statements }`
  - *A* **compound** *statement is usually put on multiple lines, with one statement per line:*
  - *Each inner statement still ends with a semicolon, but the compound statement itself does not*
- *To make an* `if` *statement "control" two or more statements, use a* **compound** *statement*
  - *by "control",* **ivan** *simply means here, "to execute"*

5.2.2. The `if` Statement (continued)

The `else` clause

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    bool result = isPrime(50);

    if (result == true) {
        printf("50 is prime");
    }
    else {
        printf("50 is not prime");
    }
}
```

- *An* `if` *statement may have an* `else` *clause:*

```c
if ( expression ) statement_1 else statement_2
```

  - *statement_2 is executed* **when** *the* **expression** *has the value* `0`

```c
if (i > j) {
    max = i;
}
else {
    max = j;
}
```

```c
if (i > j)
    max = i;
else
    max = j;
```

```c
if (i > j) max = i;
else       max = j;
```

    - *all 3 are same*

Nested `if` Statements

```c
if (i > j) {        // ppl usually do many braces
    if (i > k)
        max = i;
    else
        max = k;
}
else {              // more braces ==> more safe
    if (j > k)
        max = j;
    else
        max = k;
}
```

```c
if (i > j)          // Can do this without Braces too
    if (i > k)
        max = i;
    else
        max = k;

else                // Indentation makes it much more readable
    if (j > k)
        max = j;
    else
        max = k;
```

Cascaded `if` Statement

```c
if (n < 0)
    printf("n is less than 0\n");
else
    if (n == 0)
        printf("n is equal to 0\n");
    else
        printf("n is greater than 0\n");
```

```c
if (n < 0)
    printf("n is less than 0\n");

else if (n == 0)
    printf("n is equal to 0\n");
else
    printf("n is greater than 0\n");
```

- *A* **"cascaded"** `if` *statement is often the best way to test a series of conditions*
  - *stopping as soon as one of them is true.*

- *Although the second* `if` *statement is* **nested** *inside the first, C programmers don't usually indent it.*
  - *Instead, they align each* `else` *with the original* `if`*, like the code on the right*

5.2.3. Program : `Calculating a Broker's Comission`

When stocks are sold or purchased through a broker

- *the broker's commission is often computed using a sliding scale that depends upon the value of the stocks traded.*
- *Let's say that a broker charges the amounts shown in the following table:*

Transaction size	*Commission rate*
Under $2,500	*$30 + 1.7%*
$2,500 – $6,250	*$56 + 0.66%*
$6,250 – $20,000	*$76 + 0.34%*
$20,000 – $50,000	*$100 + 0.22%*
$50,000 – $500,000	*$155 + 0.11%*
Over $500,000	*$255 + 0.09%*

  - *The minimum charge is $39.*
  - *Our next program asks the user to enter the amount of the trade, then displays the amount of the commission:*

```
Enter value of trade: 30000
Commission: $166.00
```

    - *The heart of the program is a* **cascaded** `if` *statement*
      - *that determines which range the trade falls into.*

5.2.3. Program : `Calculating a Broker's Comission` 🟩 ANSWER

```c
/* broker.c */
#include <stdio.h>
int main(void)
{
    float value;
        printf("Enter value of trade: ");
        scanf("%f", &value);

    float comission;
         if (value < 2500.00f)                          commission = 30.00f  + .017f  * value;
    else if ((value >= 2500.00f)  && (value < 6250.00f))  commission = 56.00f  + .0066f * value;
    else if ((value >= 6250.00f)  && (value < 20000.00f))  commission = 76.00f  + .0034f * value;
    else if ((value >= 20000.00f) && (value < 50000.00f))  commission = 100.00f + .0022f * value;
    else if ((value >= 50000.00f) && (value < 500000.00f))  commission = 155.00f + .0011f * value;
    else                                                commission = 255.00f + .0009f * value;

    if (commission < 39.00f)                            commission = 39.00f;

    printf("Commission: $%.2f\n", commission);
}
```

The **cascaded** `if` statement could have been written this way instead:

```c
/* broker.c */
#include <stdio.h>
int main(void)
{
    float value;
        printf("Enter value of trade: ");
        scanf("%f", &value);

    float comission;
         if (value < 2500.00)       commission = 30.00  +  .017 * value;
    else if (value < 6250.00)       commission = 56.00  + .0066 * value;
    else if (value < 20000.00)      commission = 76.00  + .0034 * value;
    else if (value < 50000.00)      commission = 100.00 + .0022 * value;
    else if (value < 500000.00)     commission = 155.00 + .0011 * value;
    else                            commission = 255.00 + .0009 * value;

    if (commission < 39.00)         commission = 39.00;

    printf("Commission: $%.2f\n", commission);
}
```

5.2.4. 🟩 The "Dangling else" Problem

```c
if (y != 0)         // (1)
    if (x != 0)
        result = x / y;
else
    printf("Error: y is equal to 0\n");
```

- *The indentation suggests that the* `else` *clause belongs to*
  - *the* **outer** `if` *statement, i.e. belongs to* `(1)`
- *However, C follows the rule that an* `else` *clause belongs to*
  - *the* **nearest** `if` *statement that hasn't already been paired with an* `else`

- *This is called The* **"Dangling else"** *Problem*
  - *When* `if` *statements are nested, the "dangling else" problem may occur:*

The above code acts like:

```c
if (y != 0)
    if (x != 0)
        result = x / y;
    else
        printf("Error: y is equal to 0\n")
```

A correctly indented version would look like this:

```c
if (y != 0)
    if (x != 0)
        result = x / y;
    else
        printf("Error: x is equal to 0\n");
else
    printf("Error: y is equal to 0\n");
```

🟦 Or, the most ideal job to do, is to **"use more curly-braces"**

```c
if (y != 0) {
    if (x != 0)
        result = x / y;
}
else {
    printf("Error: y is equal to 0\n");
}
```

- *This would work as one would have expected in the first example right above, solely based on indentation*

5.2.5. Conditional/Ternary Operator

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int i = 0, j = 20;



    if (i < j) {
        printf(" true: i is less than j");
    }
    else {
        printf("false: i is not less than j");
    }
}
```

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int  i = 0, j = 20;

    bool b = (i < 50) ? true : false;       // Conditional Operator

    if (b == true) {
        printf(" true: i is less than j");
    }
    else {
        printf("false: i is not less than j");
    }
}
```

```c
int x = (condition_expr) ?  expr1_TRUE : expr2_FALSE;                 // yes, expr1/2 could be some random int 101

int x = (condition_expr)           ?  expr1_TRUE    : expr2_FALSE;
    if (condition_expr == true) then expr1_TRUE else expr2_FALSE;     // This is how you read it
```

- *The 3 operands*
  - `condition_expr`
  - `expr1_TRUE`
  - `expr2_FALSE`
  - *can be of any type*
- *C's* **conditional** *operator allows an* **expression** *to produce*
  - *one of two values (* `left` `right` *) of the colon* `:`
  - *depending on the value of the conditon*
- *The resulting expression is said to be a* **conditional** *expression*

---

ivan's @ page27

- *Because The* **conditional** *operator requires 3 operands,*
  - *it is often referred to as the* **ternary** *operator*
- *The conditional expression* `expr1 ? expr2 : expr3` *should be read*
  - *"* `if` *expr1*
    - `then` *expr2*
    - `else` *expr3."*
- *The expression is evaluated in stages:*
  - `expr1` *is evaluated first;*
    - *if* `expr1` $\neq 0$, *then* `expr2` *is evaluated,*
      - *and* `expr2` *'s value is the value of the entire conditional expression.*
    - *if* `expr1` $= 0$, *then* `expr3` *is evaluated*
      - *then the value of* `expr3` *is the value of the entire conditional expression.*

```c
int i = 1;
int j = 2;
int k =  i >  j ? i : j;        // k = 2
    k = (i >= 0 ? i : 0) + j;   // k = 3
```

- *The* **parentheses** *are* **necessary**
- *without them*

```c
k =  i >= 0  ?  i  :  0 + j;
k = (i >= 0) ? (i) : (0 + j);
```

  - *this is what happens.*
    - *k = 1*

---

ivan's @ page28

- *The parentheses are necessary*
  - *because the precedence of the* **conditional/ternary** *operator* $<$ *that of the other operators discussed so far,*
    - *with the exception of the* **assignment** *operators*
- **Conditional/Ternary** *Operator*
  - *Makes program shorter, but harder to* **understand**
  - *are often used in* `return` *statements:*

```c
return i > j ? i : j;
```

ivan's @ page30

- *Calls of printf can sometimes benefit from condition expressions.*

```c
if (i > j)
    printf("%d\n", i);
else
    printf("%d\n", j)
```

```c
printf("%d\n", i > j ? i : j);
```

- **Conditional/Ternary** *Operators are also common in certain kinds of* **preprocessor** *macro definitions in large software codebases.*
  - *Chapter 14.3*

5.3.1. The `switch` statement

A cascaded if statement can be used to compare an expression against a series of values:

```
if (grade == 3)
    printf("Excellent");
else if (grade == 2)
    printf("Average");
else if (grade == 1)
    printf("Poor");
else if (grade == 0)
    printf("Failing");
else
    printf("Illegal grade");
```

```
switch (grade) {
    case 3: printf("Excellent");
        break;
    case 2: printf("Average");
        break;
    case 1: printf("Poor");
        break;
    case 0: printf("Failing");
        break;
    default: printf("Illegal grade");
        break;
}
```

- *The* `switch` *statement*
  - *is an alternative*
  - *may be easier to read than a* **cascaded** `if` *statement.*
  - *are often faster than* `if` *statements.*
    - *depends on the compiler.*
- *Remember to* `break` *in each* `case`

Most common form of the `switch` statement:

```
switch ( ctrl-expression ) {
    case constant : statements
    …
    case constant : statements
    default : statements
}
```

- *The word* `switch` *must be followed by the* `ctrl` *-expression*
  - `ctrl` *-expression* **must be** `int`
    - *Characters are treated as integers in* C *and thus can be tested in* `switch` *statements*
    - *However,* `float` *and* `string` *don't qualify*

Each `case` begins with a label of the form

```
case constant-expression :
    statements;
    break;
```

- *A* **constant-expression** *is much like an ordinary expression except that it can't contain variables or function calls*
  - `5` *is a constant expression, and*
  - `5 + 10` *is a constant expression,*
  - *but* `n + 10` *isn't a constant expression*
    - *(unless n is a* **preprocessor macro** *that represents a constant)*

- *A* **constant-expression** *must evaluate to an* `int`
  - `char` *is acceptable*
  - `float` `string` *are not*

Several `case` labels may precede a group of statements:

```
switch (grade) {
    case 3:
    case 2:
    case 1:     printf("Passing");
                break;
    case 0:     printf("Failing");
                break;
    default:    printf("Illegal grade");
                break;
}
```

- *After each* **case** *label comes any number of statements*
- *No* **braces** *are required around the statements*
- *The last statement in each group is normally* `break`

To save space, several `case` labels can be put on the same line:

```
switch (grade) {

    case 3: case 2: case 1:
                printf("Passing");
                break;
    case 0:     printf("Failing");
                break;
    default:    printf("Illegal grade");
                break;
}
```

- *Duplicate* **case** *labels aren't allowed*
- *the order of* **cases** *doesn't matter*
- `default` **case** *doesn't need to come last*

- *If the* `default case` *is* **missing** *and the* `ctrl-expression` $=$ `false`
  - *then nothing happens and we simply pass on to the next statement after the entire big* `switch` *-block*

5.3.2. The `break` Statement 🟩

Now, let's take a closer look at the mysterious **break** statement. As we've seen, executing a **break** statement causes the program to **"break"** out of the **switch** statement; execution continues at the next statement after the entire big `switch` -block.

The reason that we need **break** has to do with the fact that the **switch** statement is really a form of "computed jump."
When the `ctrl-expression` is evaluated, control/**processor jumps** to the **case** label matching the value of the **switch** expression.
A **case** label is nothing more than a **marker** indicating a **position** within the **switch**.

A **case** label is nothing more than a **bookmark**

```
switch (grade) {
    case 4:     printf("Excellent");
    case 3:     printf("Good");
    case 2:     printf("Average");
    case 1:     printf("Poor");
    case 0:     printf("Failing");
    default:    printf("Illegal grade");
}
```

- **If** *grade = 3,*
- **Then** *this is printed*

```
"GoodAveragePoorFailingIllegal grade"
```

  - *Without* **break**, *processor will flow from one case into the next.*

Why?

- **If** *grade = 3*
- **Then** `switch` *sends the processor to* `case 3`
- **But**
  - *The thing is, magically after that, all the* `case 2` `case 1` `case 0` `default` **labels** *vanishes magically into thin air* 🤷‍♀️
  - *From that point onward, every statement downward is executed.*

🟩 textbook's @ page88

- *When the last statement in the* `case 3` *has been executed,*
  - *control/***processor "falls through"** *next case,*
    - *i.e. get's into the next downwards case*
  - *control/***processor** *starts executing from the first statement of the next downwards case*
  - *the case labels of all the next downwards cases are just ignored.*

🟩 ivan's @ page41

- *Without* **break** *(or some other* **jump** *statement), control will flow from one case into the next.*
- *Omitting* **break** *is sometimes done intentionally,*
  - *but it's usually just an oversight.*

```
switch (grade) {
    case 4: case 3: case 2: case 1:
            num_passing++;
            /* FALL THROUGH */
    case 0: total_grades++;
        break;
}
```

  - *It's a good idea to point out deliberate omissions of* **break** *using* **comments**
  - *Without the* **comment**,
    - *someone might* **later** *"fix the error"*
      - *by adding an unwanted* **break** *statement @ the place of the* **comment**

5.3.3. Program : `Printing a Date in Legal Form` 🟩

Contracts and other **legal** documents are often dated in the following way:

```
Dated this _________ day of _________, 20__.
```

- *Let's write a program that displays dates in this form.*

- *We'll have the user enter the date in month/day/year form, then we'll display the date in* **"legal"** *form*
- *Let's say that a broker charges the amounts shown in the following table:*

```
Enter date (mm/dd/yy): 7/19/14
Dated this 19th day of July, 2014.
```

  - *This is just a really dumb task....*
    - 🟦 *Says REYNEP*
  - **Answer on Next page**

  - *We can get* `printf()` *to do most of the formatting.*
  - *However, we're left with two problems:*
    a. how to add **"th"** *(or* **"st"** *or* **"nd"** *or* **"rd"***) to the day, and*
    b. how to print the month as a word instead of a number.
  - *Fortunately, the* **switch** *statement is ideal for solving both our problems;*
    - *we'll have one switch print the day suffix and another print the month name.*

5.3.3. Program : `Printing a Date in Legal Form` ANSWER

```c
/* Prints a date in legal form */
#include <stdio.h>

int main(void) {
    int month, day, year;

    printf("Enter date (mm/dd/yy): ");
    scanf("%d /%d /%d", &month, &day, &year);

    printf("Dated this %d", day);


    switch (day) {
        case 1: case 21: case 31:   printf("st"); break;
        case 2: case 22:            printf("nd"); break;
        case 3: case 23:            printf("rd"); break;
        default:                    printf("th"); break;
    }

    printf(" day of ");


    switch (month) {
        case 1:  printf("January");   break;
        case 2:  printf("February");  break;
        case 3:  printf("March");     break;
        case 4:  printf("April");     break;
        case 5:  printf("May");       break;
        case 6:  printf("June");      break;
        case 7:  printf("July");      break;
        case 8:  printf("August");    break;
        case 9:  printf("September"); break;
        case 10: printf("October");   break;
        case 11: printf("November");  break;
        case 12: printf("December");  break;
    }

    printf(", 20%.2d.\n", year);

    return 0;
}
```

- *Note the use of* `%.2d`
  to display the last two digits of the year.
- *If we had used* `%d` *instead,*
  - *single-digit years would be displayed incorrectly*
    - *(* `2005` *would be printed as* `205` *).*